# AI Interview Coach: Final System Architecture

One modular FastAPI app in Docker Compose. The LLM writes and grades; plain code controls the flow.

LLM step | Plain code | Data and infrastructure | Privacy-sensitive | Docker Compose boundary

*Docker Compose: web | api | postgres + pgvector | langfuse (optional)*

## BROWSER | React + TypeScript

### Resume dashboard
- Upload once; see score, fixes and likely questions

### Interview room
- Question on screen and spoken; answer by voice

### Report
- Topic scores, missed points, what to practise

### History
- Past sessions and progress over time

### Voice (in the browser)
- Speech synthesis speaks each question
- Speech recognition gives an editable transcript
- Chrome recognition sends audio to Google

### Strict mode (optional, consent)
- Camera to a local computer-vision detector
- Only timestamped flags are sent
- Video frames never leave the browser

REST + JSON, JWT

## BACKEND | FastAPI modular monolith (one app, one container)

Auth (JWT) | Validation | Per-user authorization | Rate limit + cost cap | Request ID

### Resume module
- Extract PDF text, check readable — code
- Strip personal details (PII) — code
- Parse into structured profile — LLM
- Score: keywords, sections, results — code
- Suggestions + likely questions — LLM

### Interview module (LangGraph)
- Topic picker — code
- Question generator — LLM
- Grader — LLM
- Router: follow-up, harder, next — code

*Routing is tested Python and needs no model. Follow-ups are written from the user's actual answer.*

### Knowledge module
- Retrieve chunks (pgvector) — code
- Filter by topic, role, level — code
- Source attribution — code
- Seen-question similarity check — code

*The corpus is loaded by the offline ingestion pipeline, not by users.*

### Report module
- Topic scores — code
- Missed key points — code
- Model answers from sources — LLM
- Practice plan + strict-mode flags — code

*Every model answer cites its source chunks. Flags read as possible events, not proof.*

**Background tasks (FastAPI BackgroundTasks, no Celery or Redis in version one):**
PDF parsing | embedding | pre-generating the next question while the user is still answering

## SHARED INFRASTRUCTURE

### PostgreSQL + pgvector
- users, profiles, chunks with embeddings
- sessions, turns, reports
- llm_usage for per-user cost caps

### LLM provider adapter
- Fallback chain across providers
- Caching to save free-tier quota
- Per-user usage caps
- Schema check, one retry on bad JSON

### File storage (optional)
- Docker volume for original resumes
- Retention rule + delete-my-data
- Keep only the structured profile if possible

### Observability
- Langfuse traces: prompt, chunks, cost
- Logs and error tracking
- Routing decision stored on every turn

### Ingestion pipeline (offline)
- Collect allowed sources, chunk, tag, embed, load into pgvector
- Run once per topic set, not per user

### External LLM providers (internet)
- Gemini, Groq, OpenRouter, then a local Ollama model as last resort

### Evaluation scripts (offline)
- Retrieval quality: recall@k on a labeled query set
- Grader agreement: AI scores compared with your hand scores
- Repeat rate: how often questions recur across sessions

# Design notes

How to read the architecture diagram, and why it is built this way.

## Design decisions

- **One app, not microservices.** The resume, interview, knowledge and report modules are folders inside one FastAPI app. Separate services would add networking and deployment work with no benefit at this size.
- **The LLM writes and grades; plain code decides.** Topic choice and next-step routing (follow-up, harder, next topic, finish) are tested Python. The LLM only produces questions, key points and scores as structured JSON.
- **Questions are grounded in sources.** The knowledge module retrieves chunks from pgvector and the LLM writes fresh questions from them, so every question and model answer can point to where it came from.
- **Fallback across providers, not accounts.** The adapter tries one provider, then the next (Gemini, Groq, OpenRouter, a local model), with caching and per-user caps.
- **Strict mode stays in the browser.** A local detector turns camera frames into timestamped flags. Frames are never uploaded or stored, and flags are reported as possible events, not proof of cheating.
- **Privacy by design.** Personal details are stripped before text reaches an LLM, the user can delete their data, and keeping only the structured profile (not the original PDF) is preferred.

## One interview turn

1. The topic picker chooses a topic (a resume project, a core topic or a scenario), favoring weak topics and skipping ones already asked.
2. The knowledge module retrieves the closest source chunks.
3. The question generator (LLM) writes a question plus key points, each tied to a chunk.
4. Code validates the JSON, the cited chunks, and similarity to earlier questions. One regeneration if it fails.
5. The browser speaks the question and shows it. The user answers by voice and can fix the transcript before submitting.
6. The grader (LLM) scores the answer against the key points, and code validates the output.
7. The router (plain code): under 50, one follow-up; 75 or more, a harder question; otherwise the next topic; stop at the question limit.
8. The turn, score and routing reason are saved, and the next question is pre-generated in the background.

## Main database tables

| | |
|---|---|
| **users** | Accounts and login |
| **resumes / profiles** | Structured profile, optional reference to the original file |
| **analyses** | Resume score, breakdown, suggestions, likely questions |
| **chunks** | Knowledge text, tags, source and embedding |
| **interview_sessions** | Role, settings and status |
| **turns** | Question, answer, score, routing reason, cited chunks |
| **reports** | Topic scores, missed points, practice plan, flags |
| **llm_usage** | Tokens and cost per user, for caps |

## Left out of version one

- DSA questions, because they need a code-execution sandbox. Add after the first version works.
- Azure speech. Start with browser speech and upgrade only if quality is too low.
- More roles. Adding a role later means adding topic weights and some source chunks.
- Celery, Redis and separate microservices.

## Limits to state honestly

- Chrome's speech recognition sends audio to Google, so voice input is not fully private.
- Strict mode is a deterrent, not proof. It will miss things, such as a phone held below the camera.
- Free LLM tiers change. Check the current limits before the demo.
- The resume score is a transparent fit measure with a stated formula, not a real ATS result.